

Fall 2026 Exploratory Data Analysis with R

Lecture 3: Summary Statistics and the Project

Mark Jensen
Federal Reserve Bank of Atlanta
2026-09-11

01

Recap & Agenda

01 Recap & Agenda

02 Summary Statistics

03 Measures of Location

04 Measures of Spread

05 Categorical Summary Statistics

06 Summary Statistics in Practice

07 Project Introduction

08 Recap

Quick Recap: Last Week

- Met the pipe, `%>%`, and read code top to bottom
- Learned the five core **dplyr** verbs: `select()`, `filter()`, `arrange()`, `mutate()`, `summarize()`
- Paired `group_by()` with `summarize()` to compute *by group*
- Reshaped wide data to long with `pivot_longer()`

Today: put `group_by()` and `summarize()` to work as the engine behind **summary statistics** — the first real step of data analysis.

Goals for Today

- Understand *why* summary statistics are the starting point of analysis
- Tell apart **continuous**, **discrete**, and **categorical** variables
- Measure **location** (mean, median, mode) and **spread** (sd, variance, range, IQR)
- Summarize categorical variables with frequencies, proportions, and cross-tabs
- Compute **conditional** summary statistics with `group_by()` + `summarize()`
- Meet the **course project** and how today's tools feed directly into it

Getting Started

1. Fire up R in your Posit accounts (or your IDE of choice) and start a new project
2. Upload `purchase_data.csv`
3. Prepare your environment:

```
install.packages('tidyverse')
install.packages('AER')

library(tidyverse)
library(AER)

purchase_data <- read_csv('data/purchase_data.csv')

# CPS1985 is part of the AER package, which we access through the data() function
data(CPS1985)
```

02

Summary Statistics

- 01 Recap & Agenda
- 02 Summary Statistics**
- 03 Measures of Location
- 04 Measures of Spread
- 05 Categorical Summary Statistics
- 06 Summary Statistics in Practice
- 07 Project Introduction
- 08 Recap

Why Do We Use Summary Statistics?

- Computing summary statistics is the beginning of **quantitative data analysis**.
- They **communicate** a lot of information about a dataset in a few numbers.
- Most datasets are far too large to read row by row. Summary statistics let us understand the variables and guide how we clean and manipulate them.
- They are also a **data-integrity check**. Do the numbers make sense?
 - The average of [age](#) should not be negative.
 - A strange minimum or maximum can reveal a coding mistake or a misunderstanding of how a raw variable is defined.

Variable Types

The right summary statistic depends on the *kind* of variable. Start with two **ordered** types:

1. Continuous

- Can take infinitely many values in a range
- Usually stored as numeric (`num`) or double (`dbl`)
- Ex: interest rates, wages

2. Discrete

- A finite number of values in a range
- Usually stored as numeric (`num`) or integer (`int`)
- Ex: years of education, number of complaints per day

Variable Types (cont.)

Beyond ordered variables, we have **unordered** variables — **categorical**.

■ Categorical

- Separates data into a finite number of distinct groups, or “categories”
- Can be stored as an integer (`int`), or as a string (`chr`)
- Ex: college major, zip code, region
- Note: a continuous variable cannot be categorical.

Different summary statistics apply to different variable types — this is the through-line of today’s lecture.

Two Families of Summary Statistics

For **continuous and discrete** variables:

Location — where is the center?

- Mean — `base::mean()`
- Median — `stats::median()`
- Mode — `DescTools::Mode()`

Spread — how variable is it?

- Standard deviation — `stats::sd()`
- Variance — `stats::var()`
- Range — `base::range()`
- Interquartile range — `stats::IQR()`

Plus the count of observations: `base::length()` or `base::nrow()`.

We write `package::function_name()` when it helps make clear which package a function comes from.

03

Measures of Location

- 01 Recap & Agenda
- 02 Summary Statistics
- 03 Measures of Location**
- 04 Measures of Spread
- 05 Categorical Summary Statistics
- 06 Summary Statistics in Practice
- 07 Project Introduction
- 08 Recap

Mean and Median

- The **mean** is the sum of a collection of numbers divided by how many there are. The R function is `mean()`.
- First thing to do with any new function? Find its arguments — try `?mean`.
 - `x` — the numbers to average
 - `trim` — drops a fraction of outliers before averaging (leave it alone for now)
 - `na.rm` — strip out NA values before computing? Default is `na.rm = FALSE`
- The **median** is the value that splits the data in half — 50% below, 50% above. The R function is `median()`.

```
mean(CPS1985$wage)
```

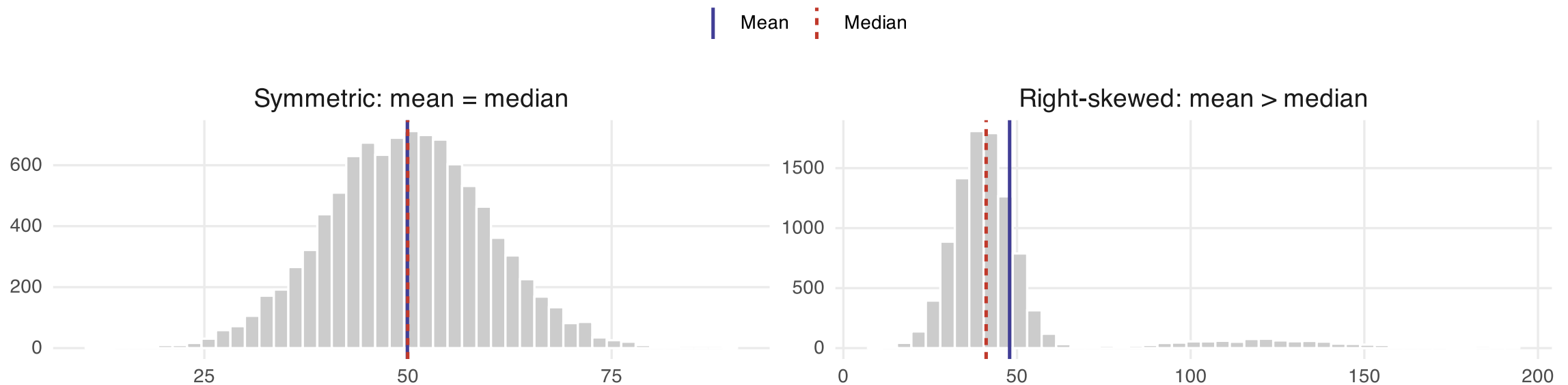
```
[1] 9.024064
```

```
median(CPS1985$wage)
```

```
[1] 7.78
```

Means, Medians, and Outliers

The gap between the mean and the median hints at the **shape** of a distribution before we ever plot it.



When the distribution is symmetric, the mean and median sit on top of each other. A long right tail pulls the **mean** toward it while the median barely moves.

Why Report the Median?

Why is “typical” income usually reported as the **median** rather than the mean?

- Add one Bill Gates to a room and the *mean* income skyrockets — the *median* hardly budes.
- The median is far less sensitive to **outliers** in either tail.
- This is exactly why the CPS wage above has a mean (9.02) noticeably higher than its median (7.78): a handful of very high earners stretch the right tail.

04

Measures of Spread

- 01 Recap & Agenda
- 02 Summary Statistics
- 03 Measures of Location
- 04 Measures of Spread**
- 05 Categorical Summary Statistics
- 06 Summary Statistics in Practice
- 07 Project Introduction
- 08 Recap

Spread: How Variable Is My Data?

Almost anything you study varies. Test scores, incomes, healthcare costs, miles per gallon — location alone doesn't tell the whole story.

Common measures of **spread**, and their R functions:

- Standard deviation — `sd()`
- Variance — `var()`
 - Variance is just standard deviation squared, so summaries usually report the standard deviation.
- Range — `range()`
- Interquartile range (IQR) — `IQR()`

```
sd(CPS1985$wage)
```

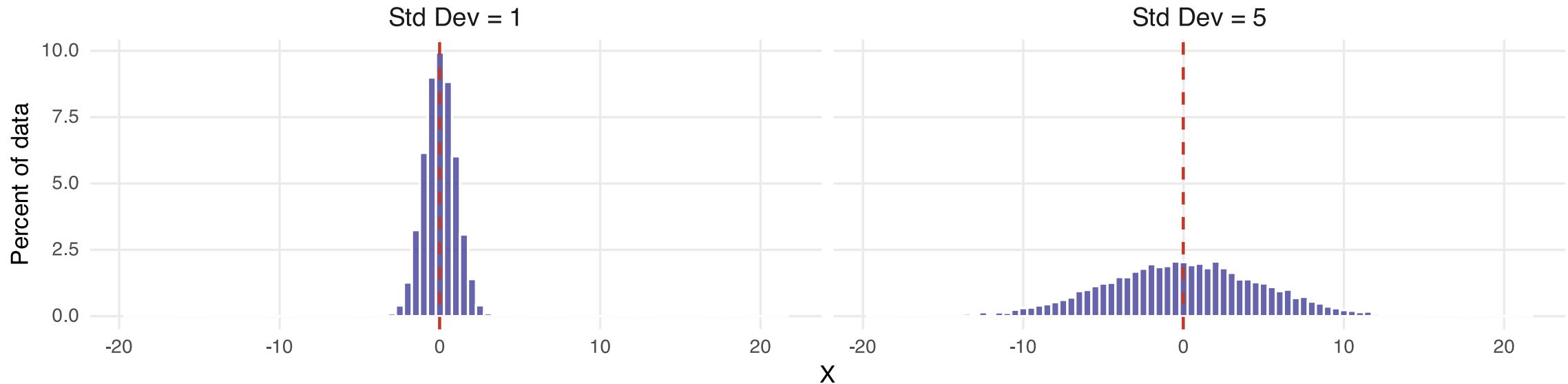
```
[1] 5.139097
```

```
IQR(CPS1985$wage)
```

```
[1] 6
```

Seeing Spread

Both samples below are drawn from a normal distribution with the **same mean**, but different standard deviations. A larger standard deviation means observations sit farther from the center — so the mean becomes a *less* useful summary.



Breakout Session 1

Task: Using `CPS1985$wage` and the functions from the last two sections, answer:

1. What are the mean and median wage? Which is larger, and what does that tell you about the shape of the wage distribution?
2. What is the standard deviation of wages? The IQR?
3. In one sentence: is the “typical” worker in this sample well described by the mean, or would you report something else?

Hint: every part here is one function call — `mean()`, `median()`, `sd()`, `IQR()`.

05

Categorical Summary Statistics

- 01 Recap & Agenda
- 02 Summary Statistics
- 03 Measures of Location
- 04 Measures of Spread
- 05 Categorical Summary Statistics**
- 06 Summary Statistics in Practice
- 07 Project Introduction
- 08 Recap

Summarizing Categorical Variables

Mean, standard deviation, and median are **not** meaningful for categorical variables:

- Average zip code? Median first name?

Instead, we describe categories with:

- **Frequencies** — raw counts per category
- **Proportions** — share of the sample in each category
- **Cross-tabulation** — counts or shares across *two* variables at once (a contingency table)

R has a special variable type for categorical data — the **factor** — which stores categories as integers mapped to labels.

A Small Categorical Dataset

We loaded `purchase_data` at the start of class: where a purchase happened, and whether it was in a Store or Online.

```
purchase_data$type <- factor(purchase_data$type,  
                             levels = c("Store", "Online"))  
str(purchase_data)
```

```
spc_tbl_ [11 × 2] (S3: spec_tbl_df/tbl_df/tbl/data.frame)  
$ store: chr [1:11] "Walmart" "Amazon" "Best Buy" "Best Buy" ...  
$ type : Factor w/ 2 levels "Store","Online": 1 2 1 1 1 2 1 2 1 2 ...  
- attr(*, "spec")=  
.. cols(  
..   store = col_character(),  
..   type = col_character()  
.. )  
- attr(*, "problems")=<pointer: 0x8bb60c690>
```

Setting the factor `levels` fixes the order categories appear in — handy for tables and, later, for plots.

Frequencies and Proportions with `tabyl()`

The `tabyl()` function from the **janitor** package counts categories and, with `adorn_percentages()`, converts them to proportions.

```
purchase_data %>% tabyl(type)
```

```
type n    percent
Store 6 0.5454545
Online 5 0.4545455
```

Cross-Tabulation

Pass **two** variables to `tabyl()` to see one broken down by the other:

```
purchase_data %>% tabyl(store, type)
```

store	Store	Online
Amazon	0	3
Best Buy	2	0
Costco	1	1
Walmart	3	1

Pipe into `adorn_percentages("all")` for shares of the whole table:

```
purchase_data %>%
  tabyl(store, type) %>%
  adorn_percentages("all")
```

store	Store	Online
Amazon	0.000000000	0.27272727
Best Buy	0.18181818	0.00000000
Costco	0.09090909	0.09090909
Walmart	0.27272727	0.09090909

Use `"row"` or `"col"` instead of `"all"` for row- or column-wise proportions.

The **tidyverse** Way

`tabyl()` is convenient, but `group_by()` + `summarize()` generalizes to anything. It's the **split-apply-combine** pattern from last week:

- **split** the data into groups
- **apply** a calculation to each group
- **combine** the results

```
purchase_data %>%  
  group_by(type) %>%  
  summarize(n = n()) %>%  
  mutate(proportion = n / sum(n))
```

```
# A tibble: 2 × 3  
  type      n proportion  
  <fct> <int>      <dbl>  
1 Store      6      0.545  
2 Online     5      0.455
```

`n()` counts rows within each group; `mutate()` then turns those counts into proportions.

06

Summary Statistics in Practice

- 01 Recap & Agenda
- 02 Summary Statistics
- 03 Measures of Location
- 04 Measures of Spread
- 05 Categorical Summary Statistics
- 06 Summary Statistics in Practice**
- 07 Project Introduction
- 08 Recap

Many Statistics at Once

We can pull single statistics — `mean()`, `median()`, `sd()`, `tabyl()` — but it's usually more informative to see several at once.

`summary()` reports a batch of summary statistics for every column you give it.

We'll explore the **Current Population Survey (CPS)**:

- **Who / Where / When:** a monthly survey of U.S. households run by the Census Bureau for the Bureau of Labor Statistics since 1940.
- **What:** demographics and labor-force characteristics.
- **Why:** it's the source of the official unemployment rate — the same `UNRATE` we pulled from FRED.

We'll use the `CPS1985` sample from the **AER** package (`data("CPS1985")`).

First Look: `glimpse()`

```
glimpse(CPS1985)
```

```
Rows: 534
Columns: 11
$ wage      <dbl> 5.10, 4.95, 6.67, 4.00, 7.50, 13.07, 4.45, 19.47, 13.28, 8.00, ...
$ education <dbl> 8, 9, 12, 12, 12, 13, 10, 12, 16, 12, 12, 12, 8, 9, 9, 12, ...
$ experience <dbl> 21, 42, 1, 4, 17, 9, 27, 9, 11, 9, 17, 19, 27, 30, 29, 37, ...
$ age       <dbl> 35, 57, 19, 22, 35, 28, 43, 27, 33, 27, 35, 37, 41, 45, 44, ...
$ ethnicity <fct> hispanic, cauc, cauc, cauc, cauc, cauc, cauc, cauc, cauc, c...
$ region    <fct> other, other, other, other, other, other, other, south, other, oth...
$ gender    <fct> female, female, male, male, male, male, male, male, male, m...
$ occupation <fct> worker, worker, worker, worker, worker, worker, worker, wor...
$ sector    <fct> manufacturing, manufacturing, manufacturing, other, other, ...
$ union     <fct> no, no, no, no, no, yes, no, no, no, no, yes, yes, no, yes, ...
$ married   <fct> yes, yes, no, no, yes, no, no, no, yes, no, yes, no, yes, n...
```

`glimpse()` shows each column's type and the number of observations — always start here.

summary() on Continuous Variables

```
CPS1985 %>%
  select(wage, education, experience) %>%
  summary()
```

wage	education	experience
Min. : 1.000	Min. : 2.00	Min. : 0.00
1st Qu.: 5.250	1st Qu.:12.00	1st Qu.: 8.00
Median : 7.780	Median :12.00	Median :15.00
Mean : 9.024	Mean :13.02	Mean :17.82
3rd Qu.:11.250	3rd Qu.:15.00	3rd Qu.:26.00
Max. :44.500	Max. :18.00	Max. :55.00

For numeric columns, `summary()` returns the min, three quartiles (including the median), the mean, and the max — location *and* spread in one call.

summary() on Factors

```
CPS1985 %>%
  select(gender, occupation, region) %>%
  summary()
```

gender	occupation	region
male :289	worker :156	south:156
female:245	technical :105	other:378
	services : 83	
	office : 97	
	sales : 38	
	management: 55	

Notice the difference: for **factors**, `summary()` returns a **frequency table** of the levels rather than a mean or median — because those wouldn't be meaningful.

A Cleaner Table with `sumtable()`

`summary()` lays statistics out in columns. The `sumtable()` function from **vtable** transposes them into rows, which is often easier to read.

```
CPS1985 %>%  
  select(wage, education, experience) %>%  
  sumtable(out = "return")
```

	Variable	N	Mean	Std. Dev.	Min	Pctl. 25	Pctl. 75	Max
1	wage	534	9	5.1	1	5.2	11	44
2	education	534	13	2.6	2	12	15	18
3	experience	534	18	12	0	8	26	55

Conditional Summary Statistics

Summary statistics of Y **conditional on** X are just summary statistics of Y computed *for each group of* X .

In other words: “summary statistics of Y conditional on X ” is a fancy way of saying **`group_by(X)` then `summarize(Y)`**.

Why care? Conditional summary statistics give the first preliminary answer to a research question. Consider:

1. Do union members earn higher wages than non-union members?
2. Do older workers earn higher wages than younger workers?

Let's use the 1985 CPS to take a first look.

Question 1: Union and Wages

Our outcome Y is `wage` (continuous); our grouping variable X is `union` (categorical). That's the ideal setup — group by the categorical variable, summarize the continuous one.

```
CPS1985 %>%  
  group_by(union) %>%  
  summarize(mean_wage = mean(wage),  
            median_wage = median(wage),  
            sd_wage = sd(wage))
```

```
# A tibble: 2 × 4  
  union mean_wage median_wage sd_wage  
  <fct>    <dbl>      <dbl>   <dbl>  
1 no      8.64        7.07    5.18  
2 yes    10.8        10      4.56
```

Union members earn more on average — in both mean and median. Which group's wages are more spread out, and what might that mean for those workers?

Question 2: When the Grouping Variable Is Continuous

Now Y is `wage` and X is `age` — but **both are continuous**. `group_by(age)` would make a group for every age, which isn't useful.

Solution: turn the continuous variable into a categorical one. Use `mutate()` with `if_else()` to flag workers above vs. below the mean age:

```
CPS1985 %>%
  mutate(age_group = if_else(age >= mean(age), "older", "younger")) %>%
  group_by(age_group) %>%
  summarize(mean_wage = mean(wage),
             median_wage = median(wage),
             sd_wage = sd(wage))
```

```
# A tibble: 2 × 4
  age_group mean_wage median_wage sd_wage
  <chr>      <dbl>      <dbl>   <dbl>
1 older      9.91        8.91    5.35
2 younger    8.34         7      4.87
```

Older workers earn more on average, with a wider spread. Why might that be?

The Key Rule

Your `group_by()` variable must be categorical.

- If it already is — like `union` — group by it directly.
- If it's continuous — like `age` — carve it into categories first with `mutate()` and `if_else()` (or `case_when()` for more than two buckets).

This one move — *bin a continuous variable, then group by it* — comes back constantly in your project work.

Breakout Session 2

Research question: Are older workers more likely to be **married** than younger workers?

Here `married` (the outcome) is *categorical* and `age` (the grouping variable) is *continuous* — so you have **two** problems to solve:

1. `group_by()` needs a categorical variable → bin `age` into "older" / "younger", as we just did.
2. You can't average a factor → turn `married` into a 0/1 indicator with `if_else(married == "yes", 1, 0)`, then take its **mean** to get the *share* married.

Task: write one piped chain that reports the share married for older vs. younger workers.

```
CPS1985 %>%
  mutate(age_group = if_else(age >= mean(age), "older", "younger"),
         married_ind = if_else(married == "yes", 1, 0)) %>%
  group_by(age_group) %>%
  summarize(share_married = mean(married_ind))
```

```
# A tibble: 2 × 2
  age_group share_married
  <chr>      <dbl>
1 older      0.776
2 younger    0.563
```

Is the *typical* older worker married? The typical younger worker?

07

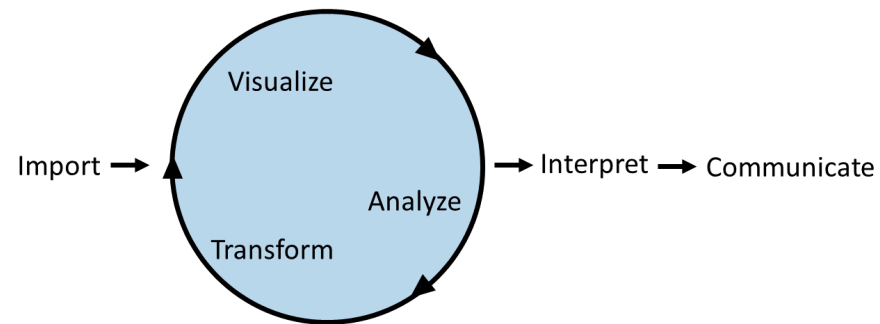
Project Introduction

- 01 Recap & Agenda
- 02 Summary Statistics
- 03 Measures of Location
- 04 Measures of Spread
- 05 Categorical Summary Statistics
- 06 Summary Statistics in Practice
- 07 Project Introduction**
- 08 Recap

The Course Project

This course builds toward a **research project worth 65% of your final grade** — the reason everything else exists. You'll use the R skills from this class to:

- explore an economic question with policy relevance,
- analyze **real-world data**, and
- work directly with an Atlanta Fed economist.



Without a completed final project, you will not pass the course.

Explore an Economic Question

Your project investigates a real economic question with policy relevance. Past projects have asked things like:

- Did the OxyContin reformulation affect heroin and fentanyl mortality?
- Do perceptions of race influence access to home mortgage lending?
- Does search intensity affect credit outcomes?

The analysis will draw on skills we build over the semester:

- summary statistics (today),
- descriptive and comparative plots,
- hypothesis tests, and
- optionally, regression.

Real Data, Real Mentors

Real-world data. You'll work with data provided through an Atlanta Fed economist. Real data comes with **no guarantees** — expect to clean and wrangle it before any analysis, exactly the tidyverse work from Lecture 2.

Your mentor and project buddy. You'll work with an Atlanta Fed economist (your mentor) and a **project buddy**. Meet with them weekly — by phone, Zoom, or otherwise. Ask lots of questions; they are an invaluable resource and are excited to work with you.

How the Project Is Graded

The 65% breaks down into two parts:

- **Six graded project updates — 20%.** Weekly, iterative assignments. Each builds on the last and asks you to incorporate the previous week's feedback, so **don't fall behind.**
- **Rough draft, in-class presentation, and final paper — 45%.**

Research is iterative. You *will* make mistakes; the point is to learn from them with feedback from your mentor and course staff along the way.

Key Project Dates (Fall 2026)

- **Project update #1 — due September 19.** The first of six weekly updates, running through October 31.
- **Rough draft — due 9:00am, November 13.** You'll get extensive feedback, including a mock grade, to guide your revision.
- **Presentations — in class, November 20.** Arrive early and prepared.
- **Final paper — due 9:00am, December 11.**

Graded on the quality of the analysis **and** how clearly you communicate it. Good R work isn't enough — the results have to be legible to a reader.

Getting Started This Week

Your first project update kicks off now. To get going:

1. Save your project proposal, data, and any documentation (see the email from course staff).
2. Create a project in your Posit Cloud workspace, import your data, and **save your script**.
3. Start exploring — this is exactly the HW you have this week:
 - Consider Who / What / When / Where / Why (read the proposal and data documentation).
 - Identify the key variables.
 - Compute summary statistics — the tools from today.

Stuck? Post on Canvas, email econ345.spelman@gmail.com, or come to office hours.

08

Recap

- 01 Recap & Agenda
- 02 Summary Statistics
- 03 Measures of Location
- 04 Measures of Spread
- 05 Categorical Summary Statistics
- 06 Summary Statistics in Practice
- 07 Project Introduction
- 08 Recap**

Recap

Today we turned last week's verbs into **summary statistics**:

- Matched the statistic to the variable type — **continuous/discrete** vs. **categorical**
- Measured **location** (mean, median, mode) and **spread** (sd, variance, range, IQR)
- Summarized categories with frequencies, proportions, and cross-tabs
- Read many statistics at once with `summary()` and `sumtable()`
- Computed **conditional** summary statistics with `group_by() + summarize()` — and learned to bin a continuous variable when we need to group by it

And we met the **project**: the same summary-statistics toolkit is exactly what your first project update asks for.

Links & Further Reading

- [R for Data Science, Ch. 3: Data Transformation](#)
- [RStudio dplyr cheat sheet](#)
- [janitor: tabyls vignette](#)
- [vtable / sumtable documentation](#)

Coming up: code hygiene and R Markdown — and your first project update.